Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Önceki dersin tekrarı: prefix kod ve Shannon-Fano
  - Bütün istatistiksel yöntemlerde üretilen kodların prefix olması gerekir; prefix olmayan kodla sıkıştırılan dosya geri açılamaz (decode edilemez).
  - Shannon-Fano'da olasılıklar sıralanır, yaklaşık eşit toplamlı iki kümeye bölünür, bir kümeye 1 diğerine 0 verilir. Entropi ile ortalama mesaj uzunluğu arasındaki fark (redundancy) ne kadar küçükse yöntem o kadar iyidir; fark büyükse veri üzerine ikinci bir sıkıştırma yöntemi daha uygulanabilir.
- Huffman kodlama yöntemi (ekrandan, 00:20:00)
  - Veri sıkıştırmada tek başına ya da başka bir yöntemin içinde bir adım olarak kullanılır. Günlük hayatta kullanılan resim formatlarında ve ZIP, WinRAR gibi arşivleyicilerde ara bir adımda Huffman kodlaması vardır.
  - Shannon-Fano'ya benzer: ikisi de kodları bir ağaç yapısında oluşturur. Fark: Huffman kodları ters olarak aşağıdan yukarıya (yapraklardan köke) doğru oluşturur, kodlar ise kökten yapraklara doğru okunur. Huffman ağacının dengeli olması gerekmez.
  - Huffman ağacı oluşturulurken iki farklı yol izlenebilir; sembollerin kodları farklı çıksa da entropi (ve ortalama kod uzunluğu) aynı çıkar.
- İki farklı Huffman ağacı örneği (ekrandan, 00:30:00)
  - Olasılıklar $a_1 = 0{,}4$, $a_2 = 0{,}2$, $a_3 = 0{,}2$, $a_4 = 0{,}1$, $a_5 = 0{,}1$. En küçük iki olasılık toplanır ve toplam sıralamayı bozmayacak şekilde diziye yerleştirilir. Toplam mevcut bir değere eşitse ya onun önüne ya arkasına konur; bir kez önüne konduysa hep önüne konmalıdır.
  - Toplamı eşit değerlerin önüne koyan birinci ağaç: kodlar `0`, `10`, `111`, `1101`, `1100`; $S = 0{,}4 \cdot 1 + 0{,}2 \cdot 2 + 0{,}2 \cdot 3 + 0{,}1 \cdot 4 + 0{,}1 \cdot 4 = 2{,}2$ bit/sembol.
  - Arkasına koyan ikinci ağaç: kodlar `11`, `01`, `00`, `101`, `100`; bit uzunlukları 2, 2, 2, 3, 3; yine $S = 2{,}2$ bit/sembol.
- Hangi kod daha iyidir: varyans (ekrandan, 00:40:00)
  - Varyans, kod uzunluklarının ortalama kod uzunluğu etrafındaki değişimidir: $\sigma^2 = \sum_i P_i(l_i - S)^2$.
  - Birinci ağaç: $0{,}4(1-2{,}2)^2 + 0{,}2(2-2{,}2)^2 + 0{,}2(3-2{,}2)^2 + 0{,}1(4-2{,}2)^2 + 0{,}1(4-2{,}2)^2 = 1{,}36$.
  - İkinci ağaç: $0{,}4(2-2{,}2)^2 + 0{,}2(2-2{,}2)^2 + 0{,}2(2-2{,}2)^2 + 0{,}1(3-2{,}2)^2 + 0{,}1(3-2{,}2)^2 = 0{,}16$.
  - Sıkıştırılmış veri dosyaya yazılacaksa varyansın önemi yoktur. Veri iletişim hattından gönderilecekse küçük varyanslı kod seçilmelidir. Gönderme hızı ile kodu üretme (çözme) hızı aynı değilse beklemeyi önlemek için buffer alanı kullanılır; varyans büyükse buffer büyük, sıfıra yakınsa küçük seçilebilir.
- Kod uzunluğu tahmini (ekrandan, 00:44:20)
  - Slaytta: a_i sembolünün kod uzunluğu $\lceil -\log_2 P_i \rceil$'ye eşit veya daha küçüktür. $P_i = 0{,}4$ için $\lceil -\log_2 0{,}4 \rceil = 2$ bit (2 ya da 1 bit ile kodlanır).
- Ortalama kod uzunluğunu ara düğümlerden hesaplamak (ekrandan, 00:46:00)
  - S, her sembolün olasılığı ile kod uzunluğunun çarpımlarının toplamıdır. Aynı değer Huffman ağacının ara düğüm olasılıklarının toplamıyla da bulunur: $0{,}2 + 0{,}4 + 0{,}6 + 1{,}0 = 2{,}2$.
- Frekanslarla Huffman örneği (ekrandan, 00:50:00 ve 00:54:30)
  - Semboller `a, e, k, l, m, z`, frekanslar sırasıyla 25, 14, 8, 7, 4, 2. Olasılık yerine

frekans kullanmak da doğrudur.
  - Adım 1: sıklıklar küçükten büyüğe sıralanır: 2, 4, 7, 8, 14, 25.
  - Adım 2: soldan başlayarak sıradaki iki değerin toplamı sıralamayı bozmayacak şekilde diziye yerleştirilir. Toplama girmiş bir değer bir daha toplanmaz. Genişletilmiş dizi: `2 4 -6 7 8 -13 14 -21 25 -35 -60`.
  - Toplamların önündeki eksi işareti, bilgisayara değerin bir ara toplam (ara düğüm) olduğunu anlatmak içindir; toplama mutlak değerle girer.
  - n sembol varsa genişletilmiş dizi $2n-1$ elemanlıdır ($n=6$ için 11). Bazı kaynaklar $2n-2$ der ve kökü diziye koymaz; hoca $2n-1$ ile tek kalan değerin kök kabul edilmesini önerdi.
- Ağacın kurulması ve kodlar (ekrandan, 01:00:00 ve 01:05:40)
  - Dizi sondan okunur. En sondaki tek değer (-60) köktür. Sonra okunan ikili onun çocuklarıdır: ilk okunan sola, ikinci okunan sağa. Negatif değer ara düğümdür, altına dal gelir; pozitif değer yapraktır.
  - Huffman ağacı ikili ağaçtır (bir düğümden en fazla iki dal çıkar). Ağacın en uzun kökten yaprağa yolu en uzun kodun, en kısa yolu en kısa kodun bit sayısını verir (burada 5 ve 1 bit).
  - Dallara etiket: soldakilerin hepsine 0, sağdakilerin hepsine 1 (ya da tamamen tersi); kural ağacın her yerinde aynı uygulanır.
  - Kökten yaprağa okunan kodlar: `a = 1`, `e = 01`, `k = 001`, `l = 0000`, `m = 00010`, `z = 00011`. Bunlar prefix koddur.
  - Kodlar yapraktan köke doğru okunursa `1`, `10`, `100`, `0000`, `01000`, `11000` çıkar; `1` ile başlayan birden çok kod olduğu için prefix değildir ve açılamaz.
- Sınıf alıştırması: frekanslar 1, 1, 1, 2, 3, 5, 6, 11 (ekrandan, 01:17:45)
  - $n=8$, genişletilmiş dizi $2n-1=15$ eleman. Ara toplamlar 2, 3, 5, 8, 11, 19, 30; kök 30.
  - Kodlar: 11 → `00`, 5 → `010`, 2 → `0110`, 1 → `0111`, 6 → `10`, 3 → `110`, 1 → `1110`, 1 → `1111`.
- Header (başlık) bilgisi (ekrandan, 01:50:00, 02:00:00 ve 01:54:05)
  - Karşı tarafın sıkıştırılmış veriyi açabilmesi için verinin başına bir header eklenir. Amaç header'ı olabildiğince küçük tutmaktır.
  - Yalnız semboller gönderilirse değişken uzunluklu kodlar okunamaz; kod uzunlukları da gönderilirse her sembol için ek bayt gerekir; frekanslar gönderilirse hem değerlerin bit uzunlukları farklıdır hem de alıcı sıralama, genişletme ve kod çıkarma işlemlerini baştan yapmak zorunda kalır.
  - Çözüm: ağacın kendisi gönderilir. Ağaç Left-Node-Right notasyonuyla dolaşılır, geçilen dalların etiketleri sırayla yazılır, her yaprakta sembol kaydedilir. Örnek ağaç için bitler `0000101111`, semboller `l m z k e a`.
  - Header: sembol sayısı için 1 bayt, 6 sembol için 6 bayt, 10 bitlik ağaç bilgisi için 2 bayt (ikinci bayt sıfırla doldurulur): toplam 9 bayt. Frekans, kod ya da kod uzunluğu gönderilmez.
  - Alıcı ağacı aynı kuralla yeniden kurar: 0 sola bir dal demektir; 0 yerine 1 gelirse bulunulan yer yapraktır, sıradaki sembol oraya konur ve bir üst düğümün sağ dalına geçilir. Sonra veri bit bit okunur; örneğin `0`, `00` kod değildir, `001` gelince `k` yazılır.
- Ağacı diziye aktarmak: düğüm adresleri (ekrandan, 02:10:00 ve 02:30:00)
  - Kök düğümün adresi 0'dır. Bir düğümün sol çocuğunun adresi kendisinin $\cdot 2+1$'i, sağ çocuğunun adresi $\cdot 2+2$'sidir.
  - Örnek ağaçta adresler: -60 → 0, -35 → 1, 25 → 2, -21 → 3, 14 → 4, -13 → 7, 8 → 8, 7 → 15, -6 → 16, 4 → 33, 2 → 34. Dizinin 5, 6 ve 9-14 gözleri boştur.
  - Kodla yapılışı: genişletilmiş dizi sondan başa okunur. Her negatif değer (ara düğüm) için indisinin $\cdot 2+1$ ve $\cdot 2+2$ değerleri peş peşe indis sütununa yazılır; pozitif değerler (yapraklar) için bir şey yapılmaz.
- İndisten kod üretme (ekrandan, 02:20:00)
  - Yalnız pozitif elemanlar (yapraklar) için yapılır; yapılan işlem indisleri yerleştirmenin tersidir. Elde edilen kod sağdan sola doğru yazılır:

```
While indis != 0 do
{
    if indis çift then
    {   print "1"
        indis = (indis-2)/2 }
```

```
    else
    {   print "0"
        indis = (indis-1)/2 }
}
```

- Sonuç tablo: 25 (indis 2) → 1, 14 (4) → 01, 8 (8) → 001, 7 (15) → 0000, 4 (33) → 00010, 2 (34) → 00011. Bitler soldan sağa yazılırsa ters kod elde edilir ve sonra çevrilmesi gerekir.
- Sembol türleri
  - Sembollerin tek karakter olması gerekmez: ikili karakterler (ab, ek) ya da kelimeler de kodlanabilir. Bu durumda header'da örneğin "kelimeler en fazla 8 harflidir, 8'er bayt oku" gibi bir bilgi verilir.
- Statik, yarı dinamik ve dinamik Huffman (ekrandan, 02:40:00)
  - **Statik:** bir derlemdeki (örneğin bir dilin bütün Wikipedia sayfaları) sembollerin kullanım sıklıklarından kodlar bir kez çıkarılır; her doküman için aynı kodlar kullanılır, dokümanın kendi sıklıklarına bakılmaz. Hızlı ve basittir ama iyi performans vermeyebilir; dokümanda geçmeyen sembollerin de kodu vardır ve bu ağacı uzatır.
  - **Yarı dinamik:** iki aşamalıdır. Önce doküman baştan sona okunup sıklıklar çıkarılır ve kodlar oluşturulur, sonra doküman yeniden okunarak kodlanır. Derste anlatılan yöntem budur.
  - **Dinamik:** kodlayıcı ve çözücü ağacı aynı anda, senkron oluşturur; metin bir kez okunur. Bir sembolün kodu dokümanın başında, ortasında ve sonunda farklı olabilir. Bunun için de varyansı küçük bir ağaç istenir.

Hocanın Özellikle Vurguladığı Kısımlar

- Prefix özelliği
  - Huffman kodları aşağıdan yukarı oluşturulur ama prefix olabilmeleri için yukarıdan aşağı (kökten yaprağa) okunmalıdır. Ters okunursa 1 ile başlayan birden çok kod olur, alıcı a mı e mi k mı olduğuna karar veremez ve bambaşka bir çıktı elde edebilir.
- Aynı değere yerleştirme kuralının tutarlılığı
  - Toplam mevcut bir değere eşitse önüne ya da arkasına konabilir; ama algoritmada bir kez önüne konduysa her seferinde önüne, arkasına konduysa her seferinde arkasına konmalıdır. Dallara 0/1 verme kuralı için de aynısı geçerlidir.
- Toplama girmiş değer bir daha toplanmaz
  - Sık yapılan hata: 2 ile 4 toplanıp, sonra 4 ile 6 toplanması. Her değer yalnız bir kez toplama girer; sıradaki ikili, henüz toplanmamış ilk iki değerdir.
- Header küçük olmalı
  - Header normal sıkıştırılmış veriden büyük olursa sıkıştırmanın anlamı kalmaz ("abesle iştigal"). En küçük header, ağacın kendisini Left-Node-Right sırasıyla göndermektir. Hoca header kısmının "altını çize çize" anlatılacağını söyledi.
- Algoritmayı bilmek
  - Hazır bir Huffman kodu bulunup kullanılabilir; ama içinde bir şeyi değiştirmek gerektiğinde nereye müdahale edileceğini bilmek için algoritmanın nasıl çalıştığını bilmek gerekir.
- İletimde küçük varyans
  - Statik ortamda (kendi diskinde bir dosyayı sıkıştırırken) hangi ağacın seçildiği önemli değildir. A noktasından B noktasına veri gönderilirken küçük varyanslı kod seçilmezse çözücü uzun bir kodu çözerken kısa kodlar birikir, buffer büyür ve gereksiz bellek kullanılır. "Büyük bir hata olmaz ama her işin bir raconu var."

Kısa Tekrar Notları

- Huffman: en küçük iki değeri topla, sıralamayı bozmadan diziye koy, toplananı tekrar toplama; eşitlikte hep aynı tarafa koy.
- Kodlar aşağıdan yukarı oluşur, yukarıdan aşağı okunur; ters okunursa prefix özelliği bozulur.
- Aynı olasılıklarla iki farklı ağaç: kodlar farklı, S ve entropi aynı.
- $\{0{,}4; 0{,}2; 0{,}2; 0{,}1; 0{,}1\}$: $S = 2{,}2$; varyanslar $1{,}36$ ve $0{,}16$.
- Varyans: $\sum P_i(l_i - S)^2$. Dosyaya yazarken önemsiz; iletimde küçük varyans → küçük buffer.

- S = ara düğüm olasılıklarının toplamı.
- Genişletilmiş dizi $2n-1$ elemanlı; eksi işaretli değerler ara toplamdır.
- Örnek kodlar: `a = 1`, `e = 01`, `k = 001`, `l = 0000`, `m = 00010`, `z = 00011`.
- Header: ağacın LNR dolaşımındaki dal etiketleri (`0000101111`) + yaprak sırasıyla semboller (`lmzkea`) + sembol sayısı → 9 bayt.
- Adresler: sol çocuk $2i+1$, sağ çocuk $2i+2$; kök 0.
- İndisten kod: çiftse `1` yaz ve $(i-2)/2$; tekse `0` yaz ve $(i-1)/2$; $i=0$ olunca dur; bitleri sağdan sola yaz.
- Statik (derlemden hazır kod), yarı dinamik (iki geçiş), dinamik (tek geçiş, kodlayıcı-çözücü senkron).

Detaylı Açıklamalar

- **İki ağacın kurulması.** Birinci ağaçta $a_4 + a_5 = 0{,}2$ elde edilir ve mevcut iki 0,2'nin önüne konur; sonra bu yeni 0,2 ile a_3'ün 0,2'si toplanıp 0,4 olur ve yine mevcut 0,4'ün önüne konur; 0,2 (a_2) ile 0,4 toplanıp 0,6, en son 0,6 ile 0,4 toplanıp 1,0 olur. Ağaç dört seviyelidir; a_1 tek bitle, a_4 ve a_5 dört bitle kodlanır. İkinci ağaçta eşit değerlerin arkasına konur: $a_4+a_5=0{,}2$ iki 0,2'nin arkasına gider, sonra $a_2+a_3=0{,}4$ mevcut 0,4'ün arkasına, sonra $0{,}2+0{,}4=0{,}6$, en son $0{,}4+0{,}6=1{,}0$. Ağaç üç seviyelidir; en uzun kod 3 bit, 1 bitlik kod yoktur. Kod uzunlukları birinci ağaçta 1, 2, 3, 4, 4; ikincide 2, 2, 2, 3, 3'tür. İkisinin de S'si 2,2'dir; entropi aynı olduğu için redundancy'leri de aynıdır (python ile $H \approx 2{,}12$ bit, $R \approx 0{,}08$ bit; derste sayısal olarak hesaplanmadı). Hoca, eşitlikte arkaya koymanın daha küçük varyanslı ağaç verdiğini; önüne koymanın varyansı büyütebildiğini belirtti.
- **Buffer örneği.** A noktasından B noktasına a_1 (5 bitlik) gönderildikten sonra arka arkaya 1 bitlik kodlar gönderilirse, B 5 bitlik kodu çözerken gelen kısa kodlar birikir. Hattın gönderme hızına çözücünün çözme hızı yetişemez; gelen bilgiler kaybolmasın diye bellekte bir buffer alanında sırasını bekler. Kod uzunlukları birbirine yakınsa (varyans küçükse) bekleme azalır.
- **Kod uzunluğu tahmini.** Hoca, olasılığı 0,4 olan bir sembol 3 bitle kodlanmışsa kodda mutlaka bir hata yapıldığını söyledi.
  - Ders sonrası not: Bu kural (kod uzunluğu $\leq \lceil -\log_2 P_i \rceil$) Huffman kodlarında her zaman geçerli değildir. Derste kurulan örnekte `m` sembolünün olasılığı $4/60 \approx 0{,}067$'dir; $\lceil -\log_2 0{,}067 \rceil = 4$ olduğu hâlde Huffman kodu `00010` 5 bittir (python ile kontrol edildi). Huffman kodu toplamda (ortalama uzunlukta) en iyidir, ama tek tek sembollerde bu sınır aşılabilir; bu nedenle sınırın aşılması tek başına hata göstergesi sayılmamalıdır.
- **Frekans örneğinde dizinin genişletilmesi.** 2 + 4 = 6 → 4 ile 7 arasına; 6 + 7 = 13 → 8 ile 14 arasına; 8 + 13 = 21 → 14 ile 25 arasına; 14 + 21 = 35 → 25'ten sonra; 25 + 35 = 60 → en sona. 60'ı toplayacak başka değer kalmayınca işlem biter. Dizide 11 eleman vardır ($2 \cdot 6 - 1$); bu sayı aynı zamanda durma noktasını verir.
- **Ağacın kurulması adım adım.** -60 kök; sondan okunan ikili (-35, 25): -35 sola, 25 sağa. -35'in çocukları (-21, 14), -21'in çocukları (-13, 8), -13'ün çocukları (7, -6), -6'nın çocukları (4, 2). Her ikilinin toplamı bir üstteki ara düğümün değerini verir; örneğin $21+14=35$, $7+6=13$. Bu yüzden toplamlar, toplandıkları yere değil sıralamayı bozmayacak yere yazılır: ağaç kurulurken bir ara düğümün çocukları dizide kendisinden hemen önce gelen ikili olur.
- **Sınıf alıştırmasının çözümü** (ekrandan, 01:17:45). Dizi `1 1 1 2 3 5 6 11`. $1+1=2$ mevcut 2'nin sağına; $1+2=3$; $2+3=5$; $3+5=8$; $5+6=11$ mevcut 11'in sağına; $8+11=19$; $11+19=30$. Kök 30: solda 19, sağda 11 (ara düğüm). 19 → 11 (yaprak) ve 8; 8 → 5 (yaprak) ve 3; 3 → 2 ve 1; sağdaki 11 → 6 (yaprak) ve 5; 5 → 3 (yaprak) ve 2; 2 → 1 ve 1. Ağırlıklı toplam $11 \cdot 2 + 6 \cdot 2 + 5 \cdot 3 + 3 \cdot 3 + 2 \cdot 4 + 3 \cdot (1 \cdot 4) = 78$ bit, sembol başına $78/30 = 2{,}6$ bit; ara düğümlerin toplamı da $30+19+11+8+5+3+2=78$'dir (python ile kontrol edildi; derste hesaplanmadı).
  - Ders sonrası not: Hoca 01:08:26'da genişletilmiş diziyi önce 17 eleman diye söyledi, sonra düzeltti: $2 \cdot 8 - 1 = 15$.

  - Ders sonrası not: Hoca 01:10:08'de yeni 3'ü mevcut 3'ün "soluna" yerleştirdiğini söyledi; ekrandaki ağaca göre (5 düğümünün altında 3 yaprağı ve 2 ara düğümü vardır) toplamlar bu alıştırmada hep eşit değerin sağına konmuştur.

- **Header seçenekleri** (01:40:31). Hoca üç sembollük bir örnekle (A: 175, B: 512, C: 1 adet; diyelim ki C = `0`, B = `10`, A = `110`) neyin gönderilebileceğini tartıştırdı. Bir öğrenci sembollerin hangi koda karşılık geldiğinin gönderilmesini önerdi. Yalnız semboller ve arka arkaya kodlar gönderilirse, kodlar değişken uzunluklu olduğu için alıcı `11`, `110` ya da `1101`'den hangisinin A'nın kodu olduğunu bilemez; sabit uzunluklu kodda "her kod 5 bit" demek yeterli olurdu. Kod uzunlukları da gönderilirse 800 farklı sembol için ek 800 bayt gerekir. Frekanslar gönderilirse değerlerin bit uzunlukları farklıdır; ya her birine 1 bayt ayrılır (1 değeri için bile) ya da büyük değerler için 2 bayt ve hangisinin kaç baytla okunacağı bilgisi eklenir; ayrıca alıcı sıralama, genişletme ve kod çıkarma işini baştan yapar.
  - Ders sonrası not: Hoca 01:43:46'da 512'nin 8 bitle yazılabildiğini söyledi; $512 = 2^9$ olduğundan 10 bit gerekir ve tek bayta sığmaz (175 ise 8 bittir). Bu, hocanın "bazı değerler bir bayta sığmaz" uyarısını destekler.

- **Header'ın çözülmesi.** `0000101111` ve `lmzkea` alınır. Dört 0 ile dört kez sola inilir; ardından 0 değil 1 geldiği için bulunulan düğüm yapraktır, `l` konur ve bir üst düğümün sağ dalına (1) geçilir. Sonra 0 ile sola inilir, ardından 1 gelir: yaprak `m`; 1 ile sağa: yaprak `z`. Bu düğümün iki dalı da bitmiştir, bir üst düğümün sağına çıkılır (1): yaprak `k`; sonra (1) yaprak `e`; sonra (1) yaprak `a`. 6 sembol yerleşince ağaç tamamdır. Hoca kuralı "0 olmadan 1 gelemez; 0 gelmeden 1 geldiyse o güzergâh bitmiştir" diye özetledi.

- **İndisten kod üretmenin çözümü.** 14 (indis 4): 4 çift → `1`, $(4-2)/2 = 1$; 1 tek → `0`, $(1-1)/2 = 0$, dur. Bitler sağdan sola yazılınca `01`. 2 (indis 34): 34 çift → `1`, 16; 16 çift → `1`, 7; 7 tek → `0`, 3; 3 tek → `0`, 1; 1 tek → `0`, 0. Sağdan sola: `00011`. Bütün kodlar python ile slayttaki tabloyla karşılaştırıldı ve tuttu.
  - Ders sonrası not: Hoca ikinci anlatımda (02:32:35 sonrası) kuralı "çift sayılarda 0, tek sayılarda 1" diye söyledi ve ekrana `0`, `10`, `110`, …, `11100` yazdı. Bu, doğru kodların tümleyenidir. Slayttaki sözde koda ve ağaca göre sağ çocuk ($2i+2$, çift) 1, sol çocuk ($2i+1$, tek) 0 etiketlidir; çift indiste `1`, tek indiste `0` yazılır.
  - Ders sonrası not: Hoca 02:28:27'deki tekrarda diziyi "iki en eksi iki" olacak şekilde genişlettiğini söyledi; derste kullanılan ve slaytta yazan kural $2n-1$'dir.

- **Shannon-Fano ile karşılaştırma.**
  - Ders sonrası not: Hoca 00:14:11'de Shannon-Fano'da olasılıkların "küçükten büyüğe" sıralandığını söyledi; önceki dersteki slayta göre büyükten küçüğe sıralanır. Ayrıca 00:17:44 ve 01:02:34'te Shannon-Fano kodlarının yapraklardan köke okunduğunu söyledi; slayttaki karşılaştırma ise kodların oluşturulma yönüyle ilgilidir: Shannon-Fano ağacı kökten başlayıp bölerek, Huffman ağacı yapraklardan başlayıp birleştirerek kurar. Önceki dersteki Shannon-Fano örneğinde de kodlar kökten yaprağa okunmuştur.

- **Statik ve yarı dinamik farkı.** Yarı dinamikte her doküman kendi kodlarını alır: bir dokümanda `a`'ya `0` verilirken başka bir dokümanda `a`'nın sayısı farklı olduğu için `11` verilebilir. Hoca, statik Huffman'ın yarı dinamikten hızlı çalıştığını, ama örneğin statikle %50 sıkıştırma elde edilirken yarı dinamikle %65'lere çıkılabileceğini söyledi.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.